

Inferring State for Real-Time Monitoring of Care Processes

Shirley A. Baffoe

Telfer School of Management
University of Ottawa
Ottawa, Canada
sbaff088@uottawa.ca

Aladdin Baarah

School of Electrical Engineering and
Computer Science
University of Ottawa
Ottawa, Canada
abaar070@uottawa.ca

Liam Peyton

School of Electrical Engineering and
Computer Science
University of Ottawa
Ottawa, Canada
lpeyton@uottawa.ca

Abstract—Patient wait times and care service times are key performance measures for care processes in hospitals. Managing the quality of care delivered by these processes in real-time is challenging. A key challenge is to correlate source medical events to infer the care process states that define patient wait times and care service times. Commercially available complex event processing engines do not have built in support for the concept of care process state. This makes it unnecessarily complex to define and maintain rules for inferring states from source medical events in a care process. In this paper, we introduce a state monitoring engine for inferring and managing states based on an application model for care process monitoring. The research is validated with a case study developed in collaboration with a large community hospital.

Index Terms—Performance measurement, healthcare, complex event processing, business process management, real-time location services, state monitoring engine, patient flow monitoring.

I. INTRODUCTION

In Canadian hospitals, as in other countries, patient wait times and care service times are key performance measures for care processes in hospitals. According to Fan et al., to empower decision makers to make timely decisions, data needs to be collected, integrated, analyzed and presented in real-time [1]. Real-time visibility into performance measures for care processes requires a data processing architecture that can aggregate disparate low level events in operational care processes to provide high level views of the states defined by the care process [1]. A key challenge in this context is to correlate source medical events to infer the care process states that define patient wait times and care service times.

The use of a Complex Events Processor (CEP) for processing events to infer medical events in real-time has been widely adopted [2], [3], and [4]. Both Business Process Management (BPM) technology [1] and Real Time Location Systems (RTLS) [5] have been used as a source of events to monitor and measure where patients are in a care process and how long they have been there [6]. Commercially available complex event processing engines do not have built in support for the concept of care process state. This makes it unnecessarily complex to define and maintain rules for inferring states from source medical events in a care process.

In this paper, we introduce a state monitoring engine for inferring and managing states based on an application model for care process monitoring. The application model explicitly captures the relationship between care process states and the metrics that measure performance for the care process, and it explicitly captures the relationship between care process states and the source medical events which are correlated to determine those states. The state monitoring engine and application model have been validated with a case study for patient flow monitoring of cardiac patients admitted to hospital through the emergency room. The case study was developed and the results of our approach were validated in collaboration with medical, nursing and IT staff at a large community hospital in Ontario, Canada.

II. BACKGROUND

Many businesses deploy performance management frameworks to improve visibility into performance measures for their organizations. In recent years, the healthcare industry has started to adopt similar frameworks. Strategic initiatives enact improvements to care processes through continuous monitoring and measurement of operational performance against targets [6]. A key component required for the development of health care performance analytics is the information technology infrastructure required to bridge the gap [7] between clinical information systems which support clinical processes and the data warehouses and reporting dashboards which support analysis of performance measures [6]. This problem is exacerbated by the complexity and diversity of low level medical event data and the wide variety of disconnected clinical information systems in a hospital. This low level operational data has to be collected, analyzed, transformed into performance measures and communicated to stakeholders to support decision making.

Traditionally, the data from existing clinical information systems is collected and processed manually or it is processed in batch using traditional Extract, Transform and Load (ETL) technology into a data warehouse [8] for off-line data analysis [7]. It typically takes days or weeks to provide the performance measures which communicate how well the healthcare process is delivering quality of care which is too late to be of use in managing day-to-day operations.

Real-time analytics involves delivering charts, metrics and alerts in real-time to support decision making [9]. According to

Chieu et. al, real-time analytics facilitates the flow of decision-critical information, thereby minimizing the time between business event (failure) happening and the time a corrective action or notification is initiated [9].

One of the technologies which can be used to provide data in support of real-time analytics is Business Process Management (BPM). BPM is defined as “the management of diverse and cross-organizational processes using methods and tools to support the design, execution, management, and analysis of business processes” [10]. A business process is a “coordinated chain of activities intended to produce a business result” [11]. BPM provides continuous monitoring of business activities and can supply real-time information about those activities.

Another technology for providing data in support of real-time analytics is Real-Time Location Systems (RTLS) which employ inexpensive Radio Frequency Identification (RFID) tags and readers to track and identify the location of objects in real-time [5]. These devices make it possible to know where patients and care providers are located in real-time.

Complex Events Processing (CEP) is a technology used to process and correlate high volumes of continuous streams of low level events in real time from disparate event sources to infer meaningful events [2]. An event-driven architecture (EDA) is defined as “a system architecture pattern whereby the behavior of the systems is characterized by the production, notification and consumption of events” [12]. The CEP processes the events provided with an EDA. The meaningful events it produces can be used to compute the performance measures used in real-time analytics. When both RTLS and BPM have been deployed in a hospital, the events from those two sources can be correlated to infer precisely the start and end times of care process states in order to compute wait times and service times [13].

III. CASE STUDY

We have conducted an extensive case study, for patient flow monitoring of cardiac patients admitted to hospital through the emergency room, in collaboration with a large community hospital in Ontario to evaluate our approach. Based on the clinical pathway for Acute Coronary Syndrome (ACS) followed by the hospital, we developed a patient flow monitoring application that could track patients through the clinical care process and monitor for bottlenecks where patients were waiting or services were taking longer than expected.

This clinical pathway was chosen because ACS patients require care within a short window of time. Monitoring and reducing the wait times for these patients is particularly important because the patient may require an urgent cardiac re-vascularisation which must be completed within 90 minutes of the event. It is important that care-givers complete the required service (e.g. consultation, lab tests, etc.) quickly so the patient can be diagnosed and receive the required treatment in time.

A key performance measure for the hospital is the overall duration of the process from triage until procedure completion. This duration is usually referred to as the time from door-to-balloon. As part of its implementation of the ACS clinical pathway, the hospital has target duration times for each step in the process from when the patient enters the emergency room

to when the procedure is performed until when the patient is discharged. A key objective of the monitoring application is to measure the durations of these low level process states and identify where bottlenecks are occurring that might impede the timely delivery of care for cardiac patients.

We modeled the flow of cardiac patients through the Emergency department (ED) and the Cardiac Care Unit (CCU) as a care process with states and created an application to measure and monitor both service and wait times in real-time. In order to measure the duration of each state, events are correlated from BPM and RTLS sources to precisely determine start and end of the key process states for the ACS clinical pathway.

Figure 1 is a sample real-time report of each patient’s state and duration along with the defined target duration for that state. The report is in reverse chronological order so the most recent state is shown first. It shows the progress of the patient through the care process from being *TRIAGED* on arrival to the ED through to the current state of *IN_BED_CW* in the cardiac ward. The patient has not yet reached the state of a cardiac procedure being performed or the state of being discharged.

Patient States

State	Start Time	End Time	Duration (mins)	Target (mins)
IN_BED_CW	2013-03-02 11:13:00.0	N/A	50	N/A
IN_TRANSPORT_CW	2013-03-02 11:02:00.0	2013-03-02 11:13:00.0	11	15
WAIT_FOR_TRANSPORT_CW	2013-03-02 10:29:00.0	2013-03-02 11:02:00.0	23	15
WAIT_FOR_BED_CW	2013-03-02 10:13:00.0	2013-03-02 10:29:00.0	26	N/A
IN_BED_ED	2013-03-02 10:12:00.0	2013-03-02 10:13:10.0	2	N/A
IN_PHYS_RE_ASSESS	2013-03-02 10:00:00.0	2013-03-02 10:12:00.0	12	N/A
WAIT_FOR_PHYS_RE_ASSESS	2013-03-02 09:40:00.0	2013-03-02 10:00:00.0	20	30
WAIT_FOR_PHYS_RE_ASSESS	2013-03-02 09:15:00.0	2013-03-02 09:40:00.0	25	30
IN_BED_ED	2013-03-02 08:15:00.0	2013-03-02 08:19:00.0	4	N/A
IN_PHYS_INIT_ASSESS	2013-03-02 08:12:00.0	2013-03-02 08:15:00.0	3	N/A
WAIT_FOR_PHYS_INIT_ASSESS	2013-03-02 08:10:00.0	2013-03-02 08:12:00.0	2	30
TRIAGED	2013-03-02 08:08:00.0	2013-03-02 08:10:00.0	2	30

Showing 1 to 12 of 12 entries

Overall Duration: 3 hours 1 mins

Figure 1: Patient State Durations versus Target Durations

The green-colored rows represent states where the measured duration is within the acceptable range (i.e. actual duration $< 2/3 * \text{target duration}$). The rows are flagged yellow where the actual duration is near the limit (i.e. $2/3 * \text{target} < \text{actual duration} < \text{target}$) and red where the actual duration exceeds the target (i.e. actual duration $> \text{target}$). Rows without color-coding have not had a time limit specified. The real-time flagging enables care providers to see where the target durations are near the limit or exceed the limit. This enables care providers to quickly identify in real-time situations where the quality of care for an individual patient may be compromised. For example, in this case not only was there an unusually long wait for a bed, but there was an unusually long wait for someone to come to transport the patient to the bed.

In addition, there is another similar looking real-time report which computes the average wait-time that all cardiac patients are currently experiencing. This allows care providers to quickly determine if the wait for transport was a onetime occurrence.

The fine-grained metrics provided by the system enable stakeholders to determine the root cause of bottlenecks. For instance, long ED patient wait durations for a bed in the cardiology ward (CW) could be because cardiac patients in the

wards are waiting to be discharged. If this were the case, we should find that the *Wait for Discharge* state is flagged red in the average wait-time report. In such cases, increasing the staff in the ED may not be effective; rather, the additional staff resources are needed in the CW to discharge the existing patients thereby freeing the beds for new admissions.

Figure 2 illustrates the system architecture that provided data integration in support of the patient flow monitoring application. Initially, in the case study, we used a commercially available complex event processing engine to correlate the events from the BPM server, RTLS server and other source systems to determine the states and wait time metrics displayed by the real-time dashboard. But, as can be seen in Figure 2, we replaced this with a State Monitoring Engine that we developed with built-in support for inferring and monitoring states in a clinical process based on an application model (shown in Figure 3 and described in section IV).

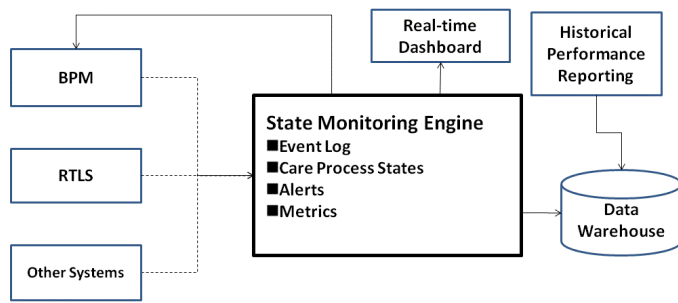


Figure 2: Architecture of State Monitoring Service

The RTLS server communicates events that identify the location of patients and care providers based on RFID readers in each room and RFID tags worn by the patients and care providers. The BPM server communicates events that indicate

when key administrative steps in the process have been performed (patient triaged, tests ordered, bed ordered etc.).

The State Monitoring Engine logs the events, infers patient states, and computes metrics related to wait times and care service times. Alerts are raised when wait times approach or exceed specified targets. A dashboard displays this data to provide a real-time view of the care process and all current patients. The state and event data from the State Monitoring Engine are logged in a relational database. The data is also transferred into a data warehouse which can be queried to generate historical performance reports in addition to those reports shown in the real-time dashboard.

Figure 3 illustrates the correlation of the main wait and service states for the ACS clinical pathway with events from either BPM or RTLS. The figure depicts a subset of the application model for monitoring the ACS clinical pathway that was developed in collaboration with hospital experts to track cardiac care patients' progress from the emergency department (ED) till they are discharged. The states after *In Procedure* are in the model, because how long one waits for a bed is affected by how long patients stay in their beds, so we model the states right up to where the patient is discharged.

The process begins when the new cardiac patient arrives in the ED for triage. The patient undergoes several wait states (*Wait for Physician Initial Assessment*, *Wait for Tests*, *Wait for Physician Re-Assessment*, *Wait for Bed*, *Wait for Procedure*, *Wait for Discharge*) and service states (*In Physician Initial Assessment*, *In Physician Re-Assessment*, *In Procedure*) as they are receiving care. These wait and service states affect the amount of time a patient spends at the hospital and can be used to pinpoint exactly where bottlenecks are occurring that affect the overall performance of the hospital.

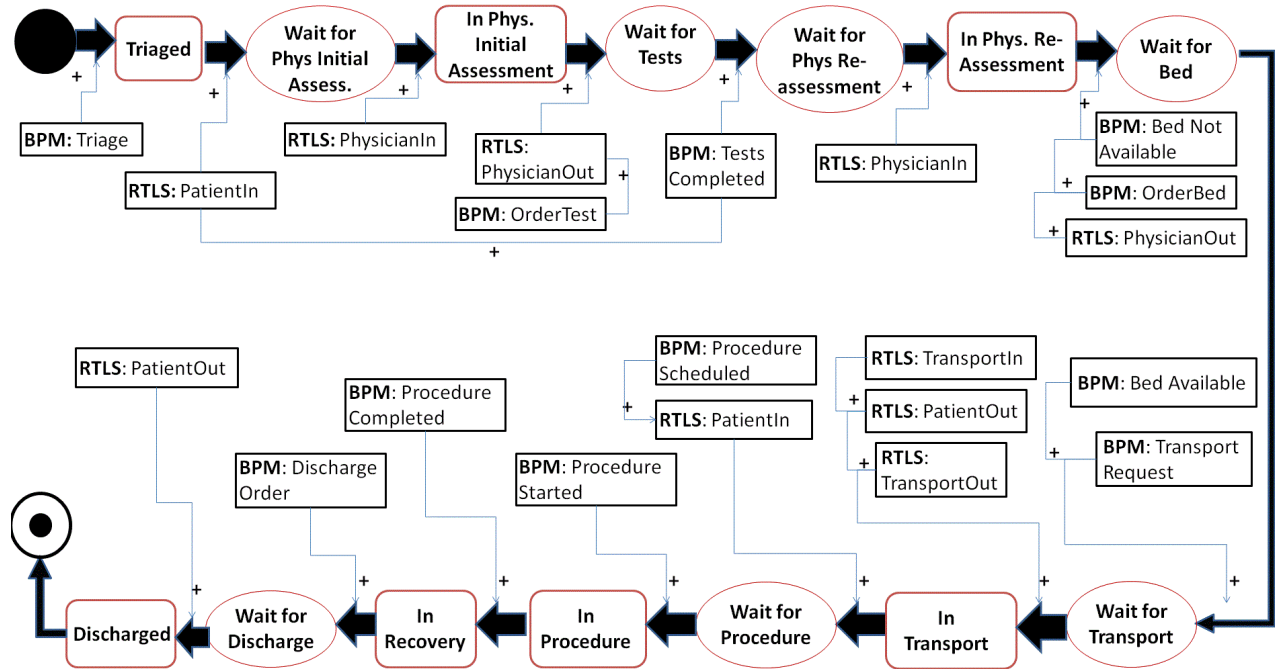


Figure 3: Care Process States for Cardiac Patient

The overall process time is also affected by how long patients are waiting for a physician, for test results, for transport, or for a bed. To accurately measure the time from door-to-balloon, the duration of wait states also need to be measured. However, neither BPM nor RTLS alone could determine the start or end of some of these wait states. For instance, the start of state, *Wait for Tests*, does not occur until both RTLS *PhysicianOut* and BPM *OrderTest* events have occurred. The physician has to both complete the assessment of the patient and finish deciding what tests to order before the wait state, *Wait for Tests*, begins. The same applies to the *Wait for Bed*, *Wait for Transport* and *Wait for Procedure* states. The correlation of source events from both RTLS and BPM and inference of process states by the State Monitoring Engine was required to accurately measure the duration of service states and wait times.

IV. STATE MONITORING ENGINE

A. Application Model

The processing required by the State Monitoring Engine in support of the real-time patient flow monitoring dashboard is defined by an application model (see Figure 4). In effect, the application model takes a simple high level state transition diagram of the key states we wish to monitor for the care process, and annotates them with links to the source events used to infer and define those states, and links to the metrics used to measure the performance of the care process against the goals for the care process.

At the center of the model is the care process and the process states we wish to measure and monitor. Those states are defined by care events that are collected from various sources (BPM, RTLS, other systems, etc.). These events and states are measured and monitored in terms of metrics that quantify the goals of the hospital for this care process (e.g. process ACS patients from *Triaged* to *In Procedure* in less than 90 minutes). Linking the performance metrics to the underlying care process states and the events which define state transitions is essential since those metrics are measured from the duration of particular states.

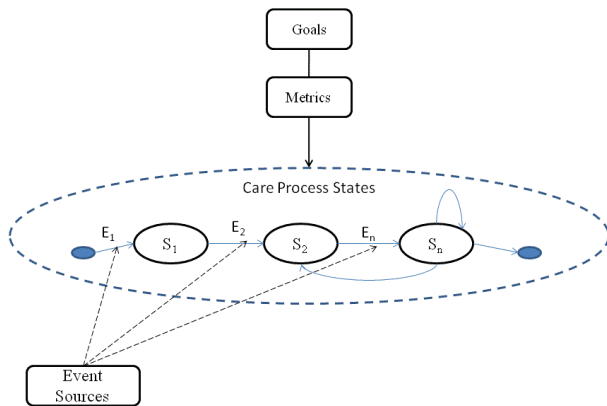


Figure 4: Application Framework Model

For instance, in Figure 3, when the current state of the cardiac patient is *In Phys. Re-Assessment*, we know that state has ended and *Wait for Bed* has started when the BPM server

indicates that a bed has been ordered, and a bed is not available, and the RTLS server indicates that the physician has left the room (even though a bed is ordered and not available, the patient is not waiting for the bed until the re-assessment has completed).

The essential design task when building a care process monitoring application is to:

- Determine what metrics best communicate the main goals for the care process (e.g. wait times, service times);
- Determine what steps and states in the care process are the key elements to measure in order to determine progress towards the ultimate goal (e.g. *Wait for Physician*, *Wait for Tests*, *Wait for Bed* in order to go from *Triaged* to *In Procedure* in less than 90 minutes);
- Determine what operational events can be correlated and aggregated from what event sources in order to infer and define care process states (BPM, RTLS, Other systems).

B. State Inference Rules

Originally, in the first phase of the case study, a commercially available CEP server was used for the patient flow monitoring application [2]. In the second phase of our case study, we replaced the CEP server with our own State Monitoring Engine which we developed. The CEP server and State Monitoring Engine performed exactly the same task of correlating events to infer and define process states. However, the CEP does not have built in support for the concept of care process state. This makes it unnecessarily complex to define and maintain rules for inferring states from source medical events in a care process. In particular, the behavior of our State Monitoring Engine is explicitly based on our application model for real-time care process monitoring. In this section, we compare the state inference rules of our State Monitoring Engine with the event inference rules in the CEP sever.

Below is a sample event inference rule for generating the event *Wait for Transport Started* in the CEP.

```
Interaction Set: Patient Wait for Transport Related by Patient.Patient_ID
In response to: TransportRequest From BPM
Immediately
  If This Event Follows WaitForBedSync
  AND
  If This Event Follows PatientAdmittedWithBed
  Then: Immediately WaitForTransportStarted
```

There are actually several rules in combination that respond to the set of events that correlate to the start of the *Wait for Transport* state. The rules are sensitive to the order events occur in, so the rule requires that the events occur in a particular order. E.g. both the events *WaitForBed* and *PatientAdmittedWithBed* must be received before the event *TransportRequest* for the rule to be triggered. If a network delay causes the *TransportRequest* event to be received before the *PatientAdmittedWithBed* event, then the *WaitForTransport* event will not be generated.

In order for the CEP to handle these anomalies, the rules have to be significantly more elaborate, so intermediate events like *WaitForBedSync* are used to coordinate. Basically, one

needs different rules to handle all possible combinations of event orderings. As well, a state is not actually inferred or managed. Instead, an event that corresponds to the start of the state is generated. In phase 1, the Real-time Dashboard had to process these start and end events and maintain its own representation of the care process as a sequence of states.

In contrast, Table 1 below shows the state inference rules used in phase 2 by the State Monitoring Engine to define state transitions from the current patient state based on correlating source events. The State Monitoring Engine rules are state-based and not event-based. In the state-based engine the current state of the patient as well as the event received determines the inferred state. While in the event-based engine used by CEP, the number of events occurring in a particular order determines the inferred state.

In the state-based approach, each state has a set of rules which are used to infer a new state from a set of received events. For instance, if the patient is in the *Triaged* state and the event *PatientIn* is received from the RTLS, the patient is transitioned into *Wait for Phys Initial Assessment* state.

Below is a sample rule for transitioning patient to the state *Wait for Transport*.

If CURRENT_STATE is *Wait for Bed*
AND EVENT *Bed Available* received
AND EVENT *TransportRequest* received
Then CURRENT_STATE is *Wait for Transport*

One advantage of having a state-based model is regardless of the order in which events are received the patient states are transitioned accurately. It is not dependent on the order in which events are received so long as all the required events are received.

Also, since each state handler subscribes to the set of events it handles, extraneous events received are ignored. For instance, if the patient is in the *Wait for Tests* state and the event *PhysicianIn* is received, the patient state will not be changed; rather, the events will be ignored (but still logged for historical analysis). This makes the State Monitoring Engine approach more robust than that of the CEP. The same situation with the CEP engine in Phase 1 resulted in the generation of a *Phys. Re-Assessment Started* event until explicit rules were

added to anticipate and handle that particular ordering of events.

As well, since the CEP only generated start and end state events in Phase 1, we were forced to perform event processing and state management in the Real-Time Dashboard (RTD). Figure 5 below, illustrates the CEP state inference model in Phase 1. In order to update a patient's state, the source event (e.g. *TransportRequest*) is first preprocessed by the CEP to generate the complex event (e.g. *WaitForTransportStarted* event). This event is then further processed by event handlers in the RTD to define and manage states and their associated metrics in support of the reports displayed by the RTD. A total of 27 CEP event rules were required for Phase 1 of our case study, and even then we could not be certain that the patient flow monitoring application would be robust in the face of all possible event sequencing that might occur. The number of event rules is proportional to the number of state transitions multiplied the number of possible event sequencings for each state transition.

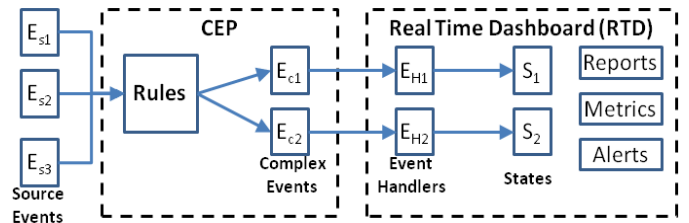


Figure 5: CEP State Inference Model in Phase 1

By contrast, in Phase 2, the State Monitoring Engine is able to manage all aspects of state and metrics using state handlers that infer and define states using the inference rules from Table 1. Figure 6 below illustrates the state inference model of the SME. The inference rules for the State Monitoring engine were much easier to articulate and manage and we were much more confident that the performance monitoring application would be robust in the face of all possible event sequencing. Only 18 state rules were needed (proportional to the number of state transitions in the care process).

TABLE 1: RULE AND PATIENT STATE MAPPING

Rules which Transition Patient State			Inferred Patient State
Current Patient State	Source Event(s)	Event Source	
Start	TriageScore	BPM	Triaged
Triaged	PatientIn	RTLS	Wait for Phys. Initial Assess.
Wait for Phys. Initial Assess.	PhysicianIn	RTLS	In Phys. Initial Assessment
In Phys. Initial Assessment	OrderTest PhysicianOut	BPM RTLS	Wait for Tests
Wait for Tests	Tests Completed	BPM	Wait for Phys. Re-assessment
Wait for Phys. Re-assessment	PhysicianIn	RTLS	In Phys. Re-Assessment
In Phys. Re-Assessment	PhysicianOut OrderBed Bed Not Available	RTLS BPM BPM	Wait for Bed
Wait for Bed	Bed Available Transport Request	BPM BPM	Wait for Transport

As an added benefit, the design and construction of the Real-time Dashboard was simplified and cleanly separated from low level event processing logic.

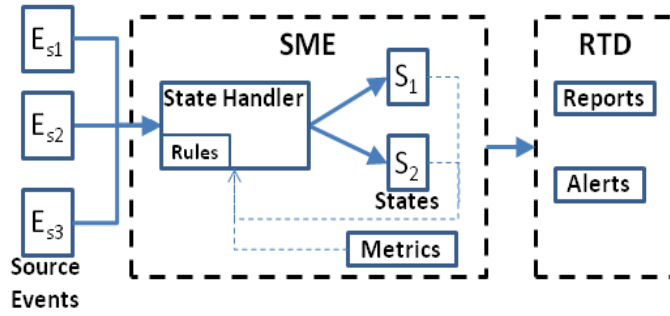


Figure 6: SME State Inference Model in Phase 2

V. EVALUATION

The entire scenario was validated using demonstrations of the ACS clinical pathway at the Ontario community hospital. The CEP engine approach was used in the first version. Some of the problems discovered with this approach led to the research into improving the model. In the new approach, the State Monitoring Engine was used to replace the CEP engine while maintaining the exact same scenario. In this section, the results of both implementations are discussed and evaluated.

Table 2 below compares the State Monitoring Engine approach with the CEP based approach. An advantage of having a state-based model over the CEP model is its rules are not dependent on the order in which events are received since the model does not depend on the order to transition the patient states. The dependency on the order in the CEP based model resulted in instability and the Patient Flow Monitoring Application where patient states were not transitioned because the events were received in the wrong order.

Another problem with the CEP rule engine was patient states were sometimes transitioned when wrong events were received. For example, if the patient is in the state *WaitForTest* and the event *PhysicianIn* is received, the patient state will be transitioned to *In Phys Initial Assessment*. In the State Monitoring Engine approach, if an erroneous event which the state handler has not subscribed to is received, the patient's state will not be transitioned. For instance, if the patient is in the *WaitForTest* state and the event *PhysicianIn* is received, the patient state will not be changed, rather, the event will be discarded.

Another advantage is the rules are more concise and precise. There are fewer rules in the new system. CEP required source event handling rules to produce higher level events and state inferring rules to update the patient events. In the new system, we only have state inferring event rules, which are less complex. In the new system each event no longer requires a handler and rules for processing, rather each state has a set of events it subscribes to. As a result, the number of handlers required for the cardiac care scenario was reduced from 30 event-based handlers to 18 state-based handlers.

Also, adding new patient events only requires updating the rules in the relevant state handler where the event is relevant. Therefore, an increase in the number of events does not correspond to more rules and handlers. The state monitoring service model is therefore more scalable. Finally the model infers events directly from the basic source events without requiring any pre-processing or higher level events.

We find the State Monitoring Engine approach to be more appropriate for the state inferring in the Patient Flow Monitoring Application. The approach requires fewer rules, defined by a clear state-based application model, to handle the scenario. Also, the approach is more robust since the rules are not dependent on event orderings or interaction. Finally, there is a clean separation between the application model and the real-time dashboard.

TABLE 2: EVALUATION OF THE STATE MONITORING MODEL

Criteria	Phase 1 (CEP)	Phase 2 (SME)
Inferring Events	Yes	No
Updating / Inferring New States	No	Yes
Application Model	No	Yes
Scalable to handle new events and states	Not really, have to re-examine all rules	Yes, simply update relevant states in state model
Number of rules and handlers for scenario	27 CEP Rules and 30 PFM event handlers	18 SME State Handlers
Metrics	Not supported	Flexibly integrated into application model
Rules Complexity	Very complex. Have to anticipate all combination of event orderings and interactions.	Less complex. Defined by a clear state-based application model. Not dependent on event orderings and interactions.
Real-Time Dashboard Complexity	Responsible for interpreting events and custom coding of states and metrics	Clean separation from real-time dashboard. Focus on reporting only against the application model.

VI. CONCLUSION

In this paper, we proposed a State Monitoring Engine for inferring and managing states based on an application model for care process monitoring. The State Monitoring Engine approach was compared with the CEP based approach used in phase 1 of our case study. The entire scenario was validated using demonstrations of the ACS clinical pathway in an Ontario community hospital. The results showed the proposed state-based approach is more robust and accurate since the application model does not depend on the order in which events are received. Fewer rules were required to handle the entire scenario and an increase in the number of events did not translate into an increase in the number of rules and handlers. This makes the new approach more reliable, scalable and easier to maintain as compared with the CEP based approach.

ACKNOWLEDGMENT

We thank IBM Canada and William Osler Health Centre for their financial support as well as their participation and collaboration in this research. This research was also funded by an NSERC strategic network grant and several MITACS internships.

REFERENCES

- [1] Y. Fan and X. Bai, "Real-Time Business Process Performance Management," *IEEE International Conference on E-Commerce Technology for Dynamic E-Business*, pp. 341 - 344, 15 September 2004.
- [2] A. Baarah and L. Peyton, "Engineering a State Monitoring Service For Real-time Patient Flow Monitoring," *Proceedings of the 9th Middleware Doctoral Symposium of the 13th ACM/IFIP/USENIX International Middleware Conference*, pp. 1-6, 2012.
- [3] T. Ku, Y. L. Zhu, K. Y. Hu and C. X. Lv, "A Novel Pattern for Complex Event Processing in RFID Applications," in *Enterprise Interoperability III*, London, Springer, 2008, pp. 595-607.
- [4] J. Boubbete-Puig, G. Ortiz and I. Medina-Bulo, "An approach of early disease detection using CEP and SOA," in *Third International Conference on Advanced Service Computing*, Rome, Italy, 2011.
- [5] W. Yao, C. Chu and Z. Li, "Leveraging complex event processing for smart hospitals using RFID," *Journal of Network and Computer Applications*, vol. 34, no. 3, pp. 799-810, May 2011.
- [6] L. Jiang, D. Barone, D. Amyot and J. Mylopoulos, "Strategic models for business intelligence," *The 30th international conference on Conceptual modeling*, pp. 429-439, 2011.
- [7] A. Mouttham, L. Peyton and C. Kuziemsky, "Leveraging Performance Analytics to Improve Integration of Care," *Proceedings of the 3rd Workshop on Software Engineering in Health Care*, pp. 56-62, May 2011.
- [8] W. H. Inmon, *Building the data warehouse*, 4th ed., Indianapolis: Wiley Publishers, 2005, p. 543.
- [9] T. C. Chieu and L. Zeng, "Real-time performance monitoring for an enterprise information management system," *IEEE International Conference on e-Business Engineering*, pp. 429-434, 22-24 October 2008.
- [10] A. Pourshahid, D. L. Amyot, L. Peyton, S. P. Ghanayati, P. Chen, M. Weiss and A. J. Forster, "Business Process Management with the User Requirements Notation," *Electronic Commerce Research*, vol. 9, no. 4, pp. 269-316, 2009.
- [11] Bruce Silver Associates, "The 2006 BPMS report: Understanding and evaluating BPM suite," BPMInstitute.org, 2006.
- [12] G. Middleton, L. Peyton, C. Kuziemsky and B. Eze, "A framework for continuous compliance monitoring of eHealth Processes," *World Congress on Privacy, Security, Trust and the Management of e-Business*, pp. 152-160, 25-27 August 2009.
- [13] A. Baarah, A. Mouttham and L. Peyton, "Improving Cardiac Patient flow based on Complex Event Processing," *Applied Electrical Engineering and Computing Technologies (AEECT)*, pp. 1-6, 6-8 December 2011.